

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

PATRON ARQUITECTONICO

Se adoptará una arquitectura RESTful sobre Node.js con Express.js, siguiendo el patrón MVC simplificado (sin capa de modelo en esta iteración) y separación de responsabilidades por capas: rutas, controladores, middlewares y configuración.

METODOLOGIA HYBRID

Dado que la metodología es híbrida, se combinarán prácticas ágiles (entrega incremental, historias de usuario) con documentación formal (Swagger/OpenAPI), permitiendo trazabilidad hacia el ticket Jira XP-9.

SEGURIDAD (DevSecOps)

1. JWT Authentication: Middleware de verificación de token en cada request protegido. Se usará la librería jsonwebtoken. El token se enviará en el header Authorization como Bearer token.
2. CORS: Configuración estricta mediante la librería cors, permitiendo solo orígenes autorizados definidos en variables de entorno.
3. Rate Limiting: Implementado con express-rate-limit para prevenir DoS/DDoS, limitando requests por IP en ventana de tiempo configurable.
4. Validación de Entrada: Uso de express-validator para sanitizar y validar parámetros de entrada.
5. Helmet: Middleware de seguridad HTTP que configura headers seguros (X-Frame-Options, CSP, HSTS, etc.).
6. Registro de Acceso: Logging estructurado con morgan para HTTP access logs y winston para logs de aplicación, incluyendo trazabilidad por request ID.

TECNOLOGIAS SELECCIONADAS

- Runtime: Node.js 20 LTS
- Framework: Express.js 4.x
- Autenticación: jsonwebtoken + bcryptjs
- Documentación: swagger-ui-express + swagger-jsdoc
- Seguridad HTTP: helmet, cors, express-rate-limit
- Validación: express-validator
- Logging: morgan + winston
- Testing: Jest + Supertest
- Linting: ESLint + Prettier
- Variables de entorno: dotenv
- Contenedorización: Docker + docker-compose
- CI/CD ready: Estructura compatible con pipelines DevSecOps

ESTRUCTURA DE CAPAS

- Config: Variables de entorno y configuraciones globales
- Middlewares: JWT, CORS, Rate Limit, Helmet, Logger, ErrorHandler
- Routes: Definición de endpoints REST
- Controllers: Lógica de negocio del endpoint
- Utils: Helpers reutilizables (logger, respuestas estándar)
- Docs: Configuración Swagger/OpenAPI
- Tests: Pruebas unitarias e integración

ENDPOINTS

- GET /api/v1/hola -> Requiere JWT válido, retorna {message: 'Hola Mundo', timestamp, requestId}
- POST /api/v1/auth/token -> Genera JWT para pruebas/desarrollo
- GET /api/v1/health -> Health check público sin autenticación
- GET /api-docs -> Documentación Swagger UI

CONSIDERACIONES FUTURAS

La estructura contempla la integración futura con MongoDB mediante Mongoose, con carpeta models y services

ya prevista en el árbol de directorios.

2. MÉTRICAS DE ITERACIÓN (QA)

- src/server.js | Estado: APROBADO | Iteraciones: 1
- src/config/app.js | Estado: APROBADO | Iteraciones: 1
- src/config/cors.js | Estado: APROBADO | Iteraciones: 1
- src/config/rateLimit.js | Estado: APROBADO | Iteraciones: 1
- src/config/swagger.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/authController.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/holaController.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/authMiddleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/errorHandler.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/requestLogger.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/validateInput.js | Estado: WARNING (Forzado) | Iteraciones: 3
- src/routes/index.js | Estado: APROBADO | Iteraciones: 1
- src/routes/authRoutes.js | Estado: APROBADO | Iteraciones: 1
- src/routes/holaRoutes.js | Estado: APROBADO | Iteraciones: 1
- src/routes/healthRoutes.js | Estado: APROBADO | Iteraciones: 1
- src/utills/logger.js | Estado: APROBADO | Iteraciones: 1
- src/utills/responseHelper.js | Estado: APROBADO | Iteraciones: 1
- src/utills/requestId.js | Estado: APROBADO | Iteraciones: 1
- src/models/.gitkeep | Estado: APROBADO | Iteraciones: 1
- src/services/.gitkeep | Estado: APROBADO | Iteraciones: 1
- tests/unit/controllers/holaController.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/middlewares/authMiddleware.test.js | Estado: APROBADO | Iteraciones: 1
- tests/integration/hola.test.js | Estado: APROBADO | Iteraciones: 1
- tests/integration/auth.test.js | Estado: APROBADO | Iteraciones: 1
- tests/integration/health.test.js | Estado: APROBADO | Iteraciones: 1
- tests/setup.js | Estado: APROBADO | Iteraciones: 1
- docs/openapi.yaml | Estado: APROBADO | Iteraciones: 1
- logs/.gitkeep | Estado: APROBADO | Iteraciones: 1
- .env.example | Estado: APROBADO | Iteraciones: 1
- .gitignore | Estado: APROBADO | Iteraciones: 1
- .eslintrc.js | Estado: APROBADO | Iteraciones: 1
- .prettierrc | Estado: APROBADO | Iteraciones: 1
- Dockerfile | Estado: APROBADO | Iteraciones: 1
- docker-compose.yml | Estado: APROBADO | Iteraciones: 1
- jest.config.js | Estado: APROBADO | Iteraciones: 1
- package.json | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 227,081

Tokens de salida (output): 90,276

Total tokens: 317,357

Horas-hombre estimadas: ~31.3 horas

Modelo utilizado: claude-sonnet-4-6